

Fundamental Biomedical Image Processing

Assignment 3

Post Date: Nov. 3, 2025; Due Date: Nov. 17, 2025

NOTE:

- Your assignments / project can be in Matlab, Python, C++.
- You can get extra bonus without using libraries.
- Deadline. The solutions to the assignments / project should be submitted by 12 pm on the date they are due. The late assignments will not be accepted.

Theoretical questions

1. A binary image contains straight lines oriented horizontally, vertically, at 45° , and at -45° . Give a set of 3 x 3 masks that can be used to detect 1-pixel breaks in these lines. Assume that the intensities of the lines and background are 1 and 0, respectively.

Mask

① Horizontal

$$\begin{bmatrix} 0 & 0 & 0 \\ 1 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}$$

原影像對上 3×3 的 Mask 後為 X

若 X 的中心 pixel 為 0 且 將 X 與 Mask 做 element-wise 相乘 結果加總為 2

② Vertical

$$\begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$$

 $\Rightarrow$ detect 1-pixel break③ $+45^\circ$

$$\begin{bmatrix} 0 & 0 & 1 \\ 0 & 0 & 0 \\ 1 & 0 & 0 \end{bmatrix}$$

④ -45°

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Programming exercises (Send the source code)

1. 程式碼為 Figure 1，輸出的結果為 Figure 2。

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3  from skimage import io
4  from skimage.morphology import disk, erosion, reconstruction
5  from skimage.util import img_as_bool
6  from skimage.measure import label, regionprops
7
8  img = io.imread("Fig1.tif")
9  print(img)
10
11  # Convert to binary: 0 is background, 1 is disk
12  Disk_binary = img > 0.5
13  binary = img_as_bool(Disk_binary)
14  # Label connected components
15  labels = label(binary)
16  print("num objects:", labels.max())
17  radii = []
18
19  for region in regionprops(labels):
20      area = region.area
21      r = np.sqrt(area / np.pi)
22      radii.append(r)
23
24  radii_sorted = sorted(radii)
25  print(radii_sorted)
26
27  # Select structuring element with radius 18(the median radius)
28  r = 18
29  selem = disk(r)
30
31  ''' Opening by Reconstruction '''
32  # Erosion
33  eroded = erosion(binary, selem)
34
35  # Reconstruction by dilation
36  reconstructed = reconstruction(eroded, binary, method='dilation')
37
38  # Plotting
39  fig, axes = plt.subplots(1, 3, figsize=(12, 4))
40  ax = axes.ravel()
41
42  ax[0].imshow(binary, cmap='gray')
43  ax[0].set_title("Original (Binary)")
44  ax[0].axis('off')
45
46  ax[1].imshow(eroded, cmap='gray')
47  ax[1].set_title("After Erosion")
48  ax[1].axis('off')
49
50  ax[2].imshow(reconstructed, cmap='gray')
51  ax[2].set_title("Opening by Reconstruction")
52  ax[2].axis('off')
53
54  plt.tight_layout()
55  plt.savefig("Fig1_result.png", dpi=300)
```

Figure 1

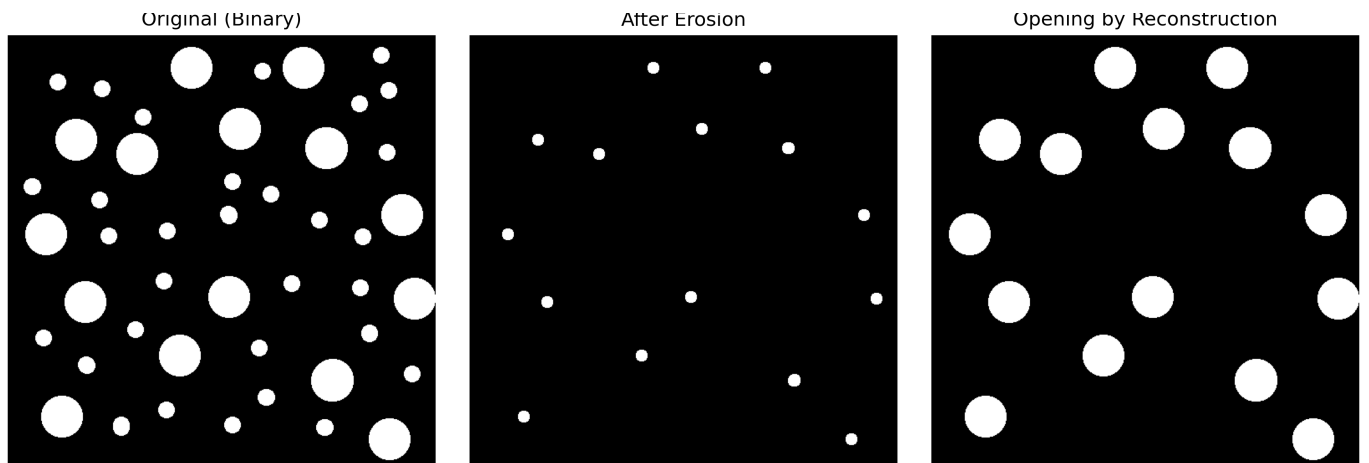


Figure 2

2.

a. 程式碼為 Figure 3，輸出的結果為 Figure 4。

```
1 import matplotlib.pyplot as plt
2 from skimage import io
3 from skimage.color import rgb2gray
4 from skimage.filters import threshold_otsu
5 from skimage.util import img_as_float
6
7 img = io.imread("Fig2.gif")
8 print(img.shape)
9 img = img[0] # gif could have multiple pages
10
11 if img.ndim == 3:
12     img_gray = rgb2gray(img)
13 else:
14     img_gray = img_as_float(img)
15
16 # Otsu's thresholding
17 t = threshold_otsu(img_gray)
18 binary = img_gray > t
19
20 # Plotting
21 plt.figure(figsize=(8, 3))
22 plt.subplot(1, 2, 1); plt.imshow(img_gray, cmap='gray'); plt.title("Gray")
23 plt.axis('off')
24 plt.subplot(1, 2, 2); plt.imshow(binary, cmap='gray'); plt.title("Binary")
25 plt.axis('off')
26 plt.tight_layout()
27 plt.savefig("Fig2a_result.png", dpi=300)
```

Figure 3

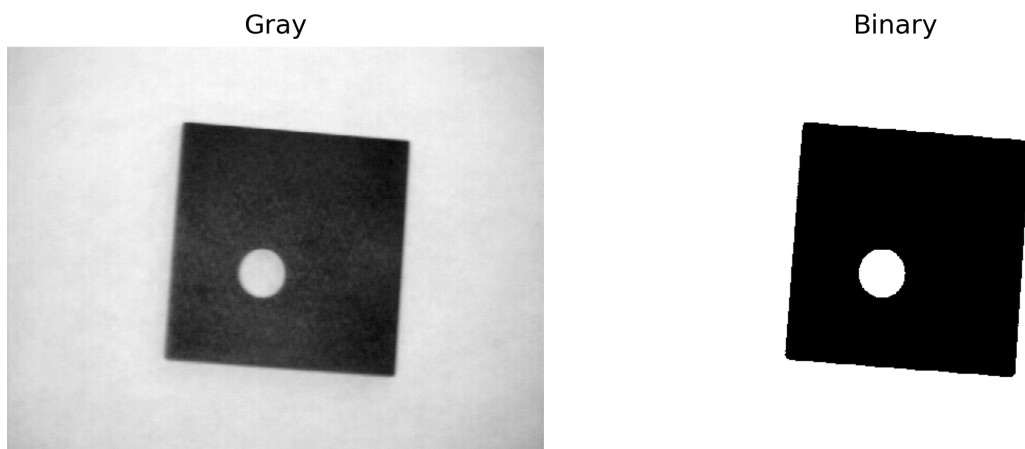


Figure 4

b. 程式碼為 Figure 5，輸出的結果為 Figure 6。

```
1  import matplotlib.pyplot as plt
2  from skimage.morphology import square, erosion, dilation
3  from skimage import io
4  from skimage.color import rgb2gray
5  from skimage.filters import threshold_otsu
6  from skimage.util import img_as_float
7
8  img = io.imread("Fig2.gif")
9  print(img.shape)
10 img = img[0] # gif could have multiple pages
11
12 if img.ndim == 3:
13     img_gray = rgb2gray(img)
14 else:
15     img_gray = img_as_float(img)
16
17 # Otsu's thresholding
18 t = threshold_otsu(img_gray)
19 binary = img_gray > t
20
21 X = binary
22
23 # structuring element, SE
24 selem = square(3) # 3x3 pixels square
25
26 # Erosion
27 X_eroded = erosion(X, selem)
28
29 # edge = X - (~X_eroded)
30 edge = X & (~X_eroded)
31
32 plt.figure(figsize=(10, 3))
33 plt.subplot(1, 3, 1); plt.imshow(X, cmap='gray'); plt.title("Binary")
34 plt.axis('off')
35 plt.subplot(1, 3, 2); plt.imshow(X_eroded, cmap='gray'); plt.title("Eroded")
36 plt.axis('off')
37 plt.subplot(1, 3, 3); plt.imshow(edge, cmap='gray'); plt.title("Edge (X - erode)")
38 plt.axis('off')
39 plt.tight_layout()
40 plt.savefig("Fig2b_result.png", dpi=300)
```

Figure 5

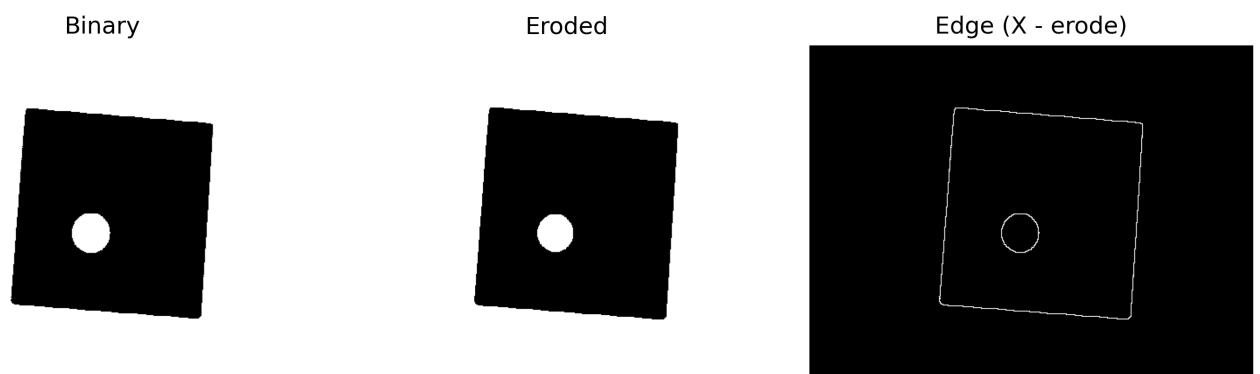


Figure 6

c. 程式碼為 Figure 7，輸出的結果為 Figure 8。

```

1 import numpy as np
2 from skimage.transform import hough_circle, hough_circle_peaks
3 from skimage.draw import circle_perimeter
4 import matplotlib.pyplot as plt
5 from skimage.morphology import square, erosion
6 from skimage import io
7 from skimage.color import rgb2gray
8 from skimage.filters import threshold_otsu
9 from skimage.util import img_as_float
10 from skimage.color import gray2rgb
11
12 img = io.imread("Fig2.gif")
13 print(img.shape)
14 img = img[0] # gif could have multiple pages
15
16 if img.ndim == 3:
17     img_gray = rgb2gray(img)
18 else:
19     img_gray = img_as_float(img)
20
21 # Otsu's thresholding
22 t = threshold_otsu(img_gray)
23 binary = img_gray > t
24
25 X = binary
26
27 # structuring element, SE
28 selem = square(3) # 3x3 pixels square
29
30 # Erosion
31 X_eroded = erosion(X, selem)
32
33 # edge = X - (~X_eroded)
34 edge = X & (~X_eroded)
35
36 # radius search range (pixel units)
37 radii = np.arange(20, 61, 2)
38
39 # Hough circle transform
40 hough_res = hough_circle(edge, radii)
41
42 # Retrieve the strongest peak (the most likely circle)
43 accums, cx, cy, radii_found = hough_circle_peaks(hough_res, radii,
44 | | | | | | | | | | total_num_peaks=1)
45
46 # Drawing the circle perimeter on the original image
47 img_rgb = gray2rgb(img_gray)
48 for center_y, center_x, radius in zip(cy, cx, radii_found):
49     rr, cc = circle_perimeter(center_y, center_x, radius)
50     img_rgb[rr, cc] = (0, 100, 0)
51
52 # Plotting
53 plt.figure(figsize=(10, 3))
54 plt.subplot(1, 3, 1); plt.imshow(edge, cmap='gray'); plt.title("Edges")
55 plt.axis('off')
56 plt.subplot(1, 3, 2); plt.imshow(img_gray, cmap='gray'); plt.title("Original gray")
57 plt.axis('off')
58 plt.subplot(1, 3, 3); plt.imshow(img_rgb); plt.title("Circle detected")
59 plt.axis('off')
60 plt.tight_layout()
61 plt.savefig("Fig2c_result.png", dpi=300)
62
63 print("Detected circle center(s) (x, y) and radius:", list(zip(cx, cy, radii_found)))

```

Figure 7

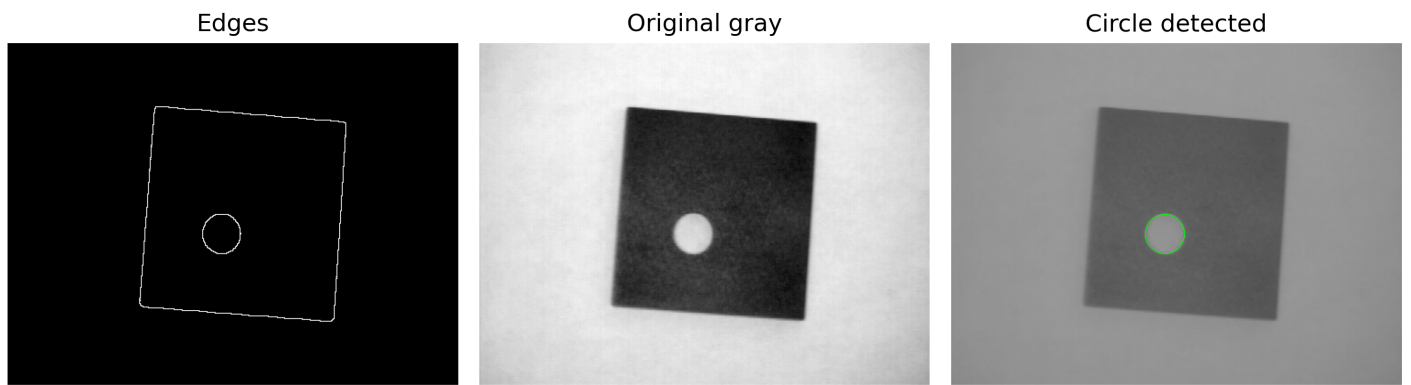


Figure 8